# Algorithms

## History, State, Behavior, Enemies of OOP

Yegor Bugayenko

Lecture #1 out of 8
80 minutes

The slidedeck was presented by the author in this YouTube Video.

All visual and text materials presented in this slidedeck are either originally made by the author or taken from public Internet sources, such as web sites. Copyright belongs to their respected authors.

- Pre-Test
- History
- Original Intent
- Object Thinking vs. Algorithms
- Enemies of Object Thinking
- Post-Test

## WARNING!

In the pursuit of academic enlightenment within this course, it is paramount to caution that the doctrines disseminated may present a potentially hazardous venture if employed in real-life software projects. This inherent risk arises from the potential incongruity with the broadly accepted canon of object-oriented programming and recognized best programming practices. If one remains resolute in their decision to adapt their coding methodologies to align with the principles propagated in this course, it would be prudent to employ a certain degree of foresight. A humorous, yet sincere suggestion, would be to secure alternate employment prior to a possible premature termination of one's current professional engagement.

Chapter #1:

# Pre-Test

[ quiz ]

https://github.com/yegor256/quiz

```
1  public class Parser {
2    private File file;
3    public synchronized void setFile(File f) {
4      file = f;
5    }
6    public synchronized File getFile() {
7      return file;
8    }
9    public String getContent() throws IOException {
10     // Read the content of the file
11     // and return it.
12   }
13   public String getContentWithoutUnicode() throws IOException {
14     // Read the file and filter out symbols
15     // that are not UTF-8 compliant.
16   }
17   public void saveContent(String content) {
18     // Save the "content" to the file.
19   }
20 }
```

[ quiz ]

Chapter #2:

# History

# Who Started It?



Ivan Sutherland's seminal **Sketchpad** application was an early inspiration for OOP, created between 1961 and 1962 and published in his Sketchpad Thesis in 1963. Any object could become a "master," and additional instances of the objects were called "occurrences". Sketchpad's masters share a lot in common with JavaScript's prototypal inheritance. (c) Wikipedia

# Who Invented Objects, Classes, and Inheritance?




**Simula** was developed in 1965 at the Norwegian Computing Center in Oslo, by Ole-Johan Dahl and Kristen Nygaard. Like Sketchpad, Simula featured objects, and eventually introduced classes, class inheritance, subclasses, and virtual methods. (c) Wikipedia

# Simula-67: Sample Code

```
1  Class Figure;
2    Virtual: Real Procedure area Is Procedure area;;
3  Begin
4  End;
5  Figure Class Circle (c, r);
6    Real c, r;
7  Begin
8    Real Procedure area;
9    Begin
10     area := 3.1415 * r * r;
11   End;
12 End;
```

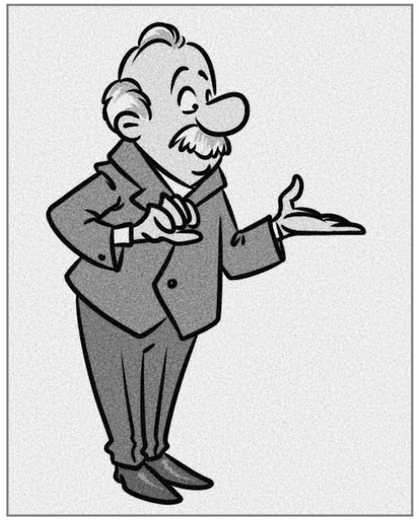
# Who Coined the "OOP" Term?



**Smalltalk** was created in the 1970s at Xerox PARC by Learning Research Group (LRG) scientists, including Alan Kay, Dan Ingalls, Adele Goldberg, Ted Kaehler, Diana Merry, and Scott Wallace. (c) Wikipedia

# Smalltalk: Sample Code

```
1  Object subclass: Account [
2      | balance |
3      Account class >> new [
4          | r |
5          r := super new. r init. ^r
6      ]
7      init [ balance := 0 ]
8  ]
9  Account extend [
10     deposit: amount [ balance := balance + amount ]
11 ]
12 a := Account new
13 a deposit: 42
```



"Everyone will be in a favor of OOP. Every manufacturer will promote his products as supporting it. Every manager will pay lip service to it. Every programmer will practice it (differently). And no one will know just what it is."

— Tim Rentsch. "Object Oriented Programming". In: *ACM SIGPLAN Notices* 17.9 (1982), pp. 51–57. DOI: 10.1145/947955.947961

# Who Made It All Popular?



**C++** was created by Danish computer scientist Bjarne Stroustrup in 1985, by enhancing the C language with Simula-like features. C was chosen because it was general-purpose, fast, portable and widely used.

You may enjoy watching this one-hour dialog of Dr. Stroustrup and me.

# C++: Sample Code

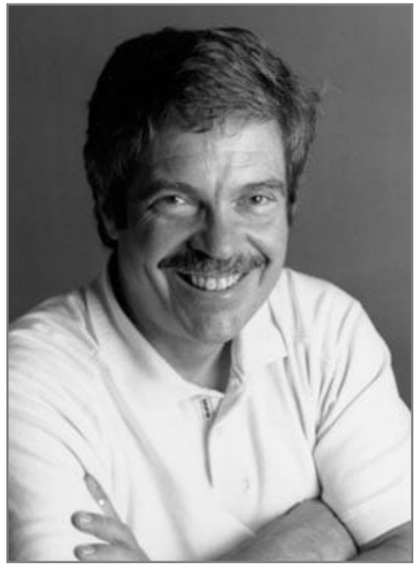
```
1 class Figure {
2   virtual float area() = 0;
3 };
4 class Circle : public Figure {
5   Circle(float c, float r) : c(c), r(r) {};
6   float area() { return 3.1415 * r * r; };
7 private:
8   float c, r;
9 };
```



Ole Lehrmann Madsen

"There are as many definitions of OOP as there are papers and books on the topic."

— Ole Lehrmann Madsen and Birger Møller-Pedersen. "What Object-Oriented Programming May Be — And What It Does Not Have to Be". In: *Proceedings of the European Conference on Object-Oriented Programming*. Springer. 1988, pp. 1–20. DOI: 10.1007/3-540-45910-3_1



"I made up the term 'object-oriented,' and I can tell you I didn't have C++ in mind."

— Alan Kay. *The Computer Revolution Hasn't Happened yet.* 1997. DOI: 10.1007/978-3-322-89884-5_3

There was an interesting debate between Alan Kay and a few readers of my blog, in the comments section under this blog post: Alan Kay Was Wrong About Him Being Wrong [2].

# What Happened Later?

C++ was released in 1985. And then...

Erlang (1986)
Eiffel (1986)
Self (1987)
Perl (1988)
Haskell (1990)
Python (1991)
Lua (1993)
JavaScript (1995)
Ruby (1995)
Java (1995)
PHP3 (1998)
C# (2000)
Go (2009)
Rust (2010)
Swift (2014)
EO (2016)

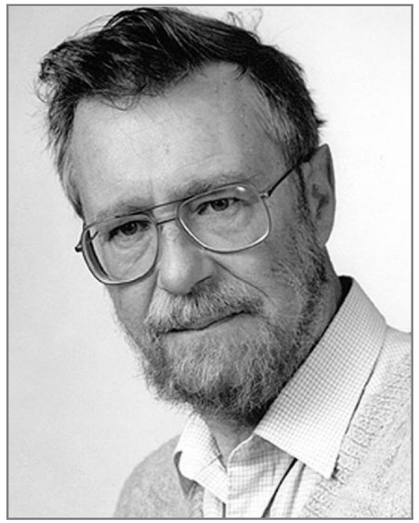


"There is no uniformity or an agreement on the set of features and mechanisms that belong in an OO language as the paradigm itself is far too general."

— Oscar Nierstrasz. *A Survey of Object-Oriented Concepts*. Citeseer, 1989

# Incomplete List of OOP Features, ... So Far:

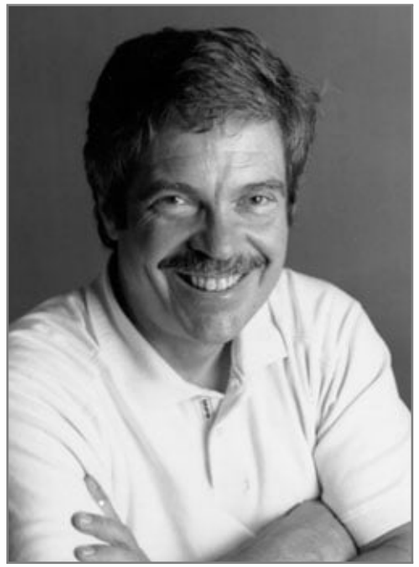
- Polymorphism
- Nested Objects
- Traits
- Templates
- Generics
- Invariants
- Classes
- NULL
- Exceptions
- Operators
- Methods
- Static Blocks
- Virtual Tables
- Coroutines
- Monads
- Algebraic Types
- Annotations
- Interfaces
- Constructors
- Destructors
- Lifetimes
- Volatile Variables
- Synchronization
- Macros
- Inheritance
- Overloading
- Tuple Types
- Closures
- Access Modifiers
- Pattern Matching
- Enumerated Types
- Namespaces
- Modules
- Type Aliases
- Decorators
- Lambda Functions
- Type Inference
- Properties
- Value Types
- Multiple Inheritance
- Events
- Callbacks
- NULL Safety
- Streams
- Buffers
- Iterators
- Generators
- Aspects
- Anonymous Objects
- Anonymous Functions
- Reflection
- Type Casting
- Lazy Evaluation
- Garbage Collection
- Immutability



Edsger Dijkstra

"Object oriented programs are offered as alternatives to correct ones... Object-oriented programming is an exceptionally bad idea which could only have originated in California."

— Bob Crawford. *A Series of Articles About Object Oriented Programming.* TUG LINES, 1989



Alan Kay

"I made up the term 'object-oriented,' and I can tell you I didn't have C++ in mind."

— Alan Kay. *The Computer Revolution Hasn't Happened yet*. 1997. DOI: 10.1007/978-3-322-89884-5_3



Linus Torvalds

"C++ is a horrible language. . . C++ leads to really, really bad design choices. . . In other words, the only way to do good, efficient, and system-level and portable C++ ends up to limit yourself to all the things that are basically available in C."

— Esther Schindler. *Linus Torvalds: Why C++ Sucks.* https://jttu.net/schindler2007. [Online; accessed 08-04-2025]. CIO, 2007



Jeff Atwood

"OO seems to bring at least as many problems to the table as it solves."

— Jeff Atwood. *Your Code: OOP or POO?*. https://jttu.net/atwood2007. [Online; accessed 08-04-2025]. 2007



Rich Hickey

"I think that large object-oriented programs struggle with increasing complexity as you build this large object graph of mutable objects. You know, trying to understand and keep in your mind what will happen when you call a method and what will the side effects be."

— Rich Hickey. *SE Radio 158: Rich Hickey on Clojure*. https://jttu.net/hickey2010. [Online; accessed 08-04-2025]. 2010

> The complexity of object-oriented code remains its primary drawback



"Reading an OO code you can't see the big picture and it is often impossible to review all the small functions that call the one function that you modified."

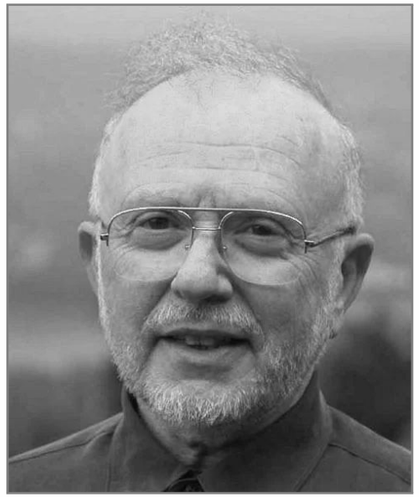
— Asaf Shelly. *Flaws of Object Oriented Modeling*. https://jttu.net/shelly2015flaws. [Online; accessed 15-03-2016]. 2015

> Thus, we don't know anymore what exactly is object-oriented programming, and whether it helps us write better code.

You can find more quotes in this blog post of mine: What's Wrong With Object-Oriented Programming? [3]

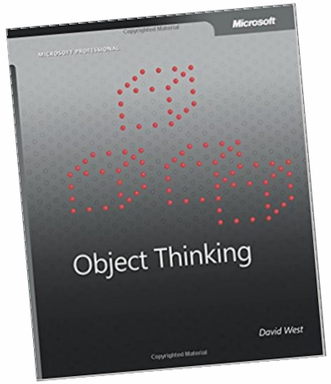
Chapter #3:

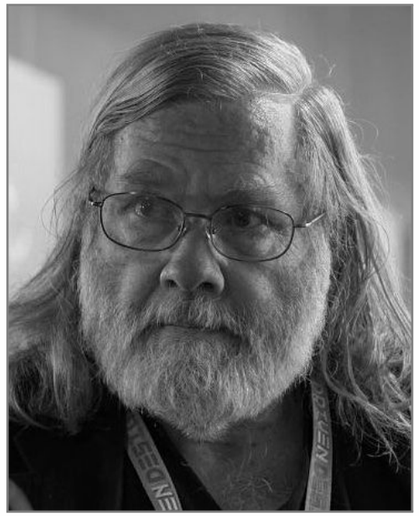
# Original Intent



"It is almost always incorrect to begin the decomposition of a system into modules on the basis of a flowchart. We propose instead that one begins with a list of difficult design decisions or design decisions which are likely to change. Each module is then designed to hide such a decision from the others."

— David Lorge Parnas. "On the Criteria to Be Used in Decomposing Systems into Modules". In: *Communications of the ACM* 15.12 (1972), pp. 1053–1058. DOI: 10.1145/361598.361623



"The contemporary mainstream understanding of objects (which is not behavioral) is but a pale shadow of the original idea and antithetical to the original intent."

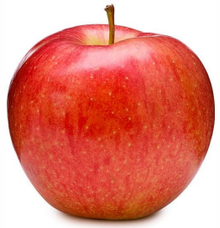
— David West. *Object Thinking*. Pearson Education, 2004. DOI: 10.5555/984130

You may enjoy watching our conversation with Dr. David West, video-recorded and published on YouTube: part I and part II.

> A system is a composition of objects that are abstractions, which hide data and expose behavior*

* This is how I understand the original intent.

# 1) What Is an “Abstraction”?

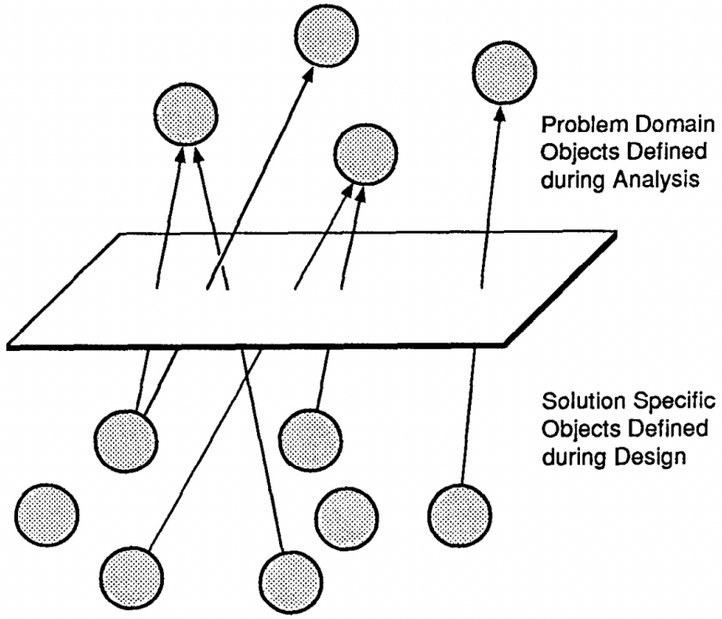


- Color: red
- Weight: 120g
- Price: $0.99



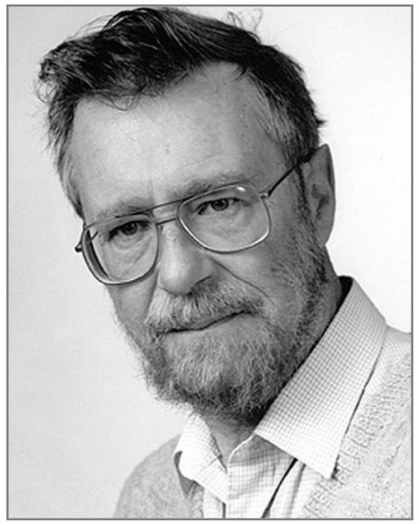
```
var file = {
  path: '/tmp/data.txt',
  read: function() { ... },
  write: function(txt) { ... }
}
```

We deal with an abstraction as if it was a real thing, but eliminating unnecessary details. We do `file.read()` instead of “open file handler for data.txt, read byte by byte, store in byte buffer, wait for the end of file, and return the result.”




"Object-oriented design is first concerned with entities—things. These things may be tangible objects such as traffic lights, chairs, or airplanes. The entities may be abstract concepts such as roles, interactions, or incidents. From a design perspective, objects model the entities in the application domain."

Source: Tim Korson and John D. McGregor. "Understanding Object-Oriented: A Unifying Paradigm". In: *Communications of the ACM* 33.9 (1990), pp. 40–60. DOI: 10.1145/83880.84459



"The effective exploitation of his powers of abstraction must be regarded as one of the most vital activities of a competent programmer... By suitable application of our powers of abstraction, the intellectual effort required to conceive or to understand a program need not grow more than proportional to program length."

— Edsger W. Dijkstra. "The Humble Programmer". In: *Communications of the ACM* 15.10 (1972), pp. 859–866. DOI: 10.1145/355604.361591

# How Many Abstractions Are Needed?

```
int area(x1, y1, x2, y2) {
  int w = x2 - x1;
  if (w < 0) { w = w * -1; }
  int h = y2 - y1;
  if (h < 0) { h = h * -1; }
  return w * h;
}
```

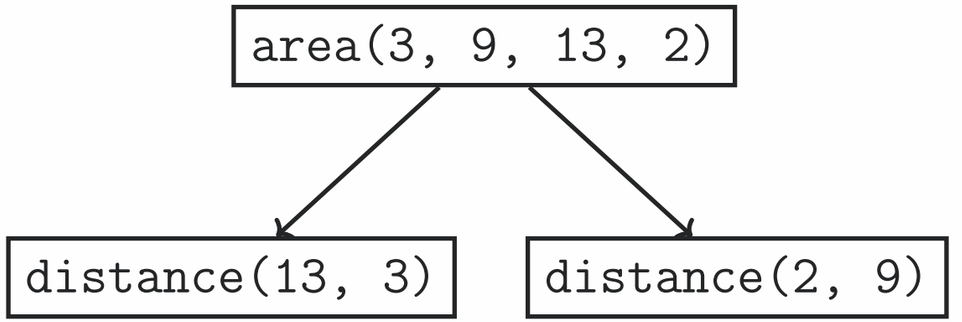
```
int distance(left, right) {
  int d = right - left;
  if (d < 0) { d = d * -1; }
  return d;
}

int area(x1, y1, x2, y2) {
  return distance(x2, x1)
    * distance(y2, y1);
}
```

There are two abstractions at the right snippet ("area" and "distance"), while only one abstraction at the left one (just "area").

# Levels of Abstraction

```
1  int distance(left, right) {
2    int d = right - left;
3    if (d < 0) { d = d * -1; }
4    return d;
5  }
6
7  int area(x1, y1, x2, y2) {
8    return distance(x2, x1)
9      * distance(y2, y1);
10 }
```




Higher level abstractions must not know and/or rely on semantics of lower level abstractions.

# 2) What Is "Data Hiding"?

```
1 f = new File("/tmp/data.txt");
2 // The data escapes the object! :(
3 p = f.getPath();
4 FileUtils.deleteFile(p);
```

```
1 f = new File("/tmp/data.txt");
2 // The boolean data escapes too :)
3 done = f.delete();
4 assert(done);
```

Obviously, some data must escape your objects.

# 3) What Is “Behavior Exposing”?

This is so called “anemic” object:

```
var user = {
  login: 'jeff',
  password: 'swordfish',
  age: 32
}
function print(u) {
  console.log(`Hello, ${u.login},
    you are ${u.age} today!`);
}
print(user);
```

This object is “alive”:

```
var user = {
  login: 'jeff',
  password: 'swordfish',
  age: 32,
  print: function() {
    console.log(`Hello, ${this.
  login},
      you are ${this.age} today!`);
  }
}
user.print();
```

# An Object as a Function

```
int distance(left, right) {
  int d = right - left;
  if (d < 0) { d = d * -1; }
  return d; }
int area(x1, y1, x2, y2) {
  return distance(x2, x1)
    * distance(y2, y1); }
```

```
class Distance {
  private int r; private int l;
  Distance(a, b) { l = a; r = b; }
  int value() {
    int d = r - l;
    if (d < 0) { d = d * -1; }
    return d; } }
int area(x1, y1, x2, y2) {
  return new Distance(x2, x1).value
  ()
    * new Distance(y2, y1).value();
   } }
```

The Java object `Distance` on the right snippet is semantically equivalent to the C function `distance()` on the left one.

# Identity, State, Behavior

```
class Circle {
  private float radius;
  Circle(float r) {
    radius = r; }
  float getRadius() {
    return radius; }
  void setRadius(float r) {
    radius = r; }
  float area() {
    return 3.14 * radius * radius;
  }
}
```

```
// Identity:
c1 = new Circle(42.0);
c2 = new Circle(42.0);
c1 != c2;
// State:
c1 = new Circle(42.0);
c2 = new Circle(42.0);
c1.getRadius() == c2.getRadius();
// Behavior:
c1 = new Circle(42.0);
c2 = new Circle(-42.0);
c1.area() == c2.area();
```

# State vs. Behavior

```
class Circle {
  private float r;
  void setR(float r) { this.r = r;
  }
  float getR() { return this.r; }
}
class FigureUtils {
  static float area(Circle c) {
    return 3.14 * c.getR() * c.getR
  ();
  }
}
Circle c = new Circle();
c.setR(42.0);
float s = FigureUtils.area(c);
```

```
class Circle {
  private float r;
  Circle(float r) { this.r = r; }
  float area() {
    return 3.14 * this.r * this.r;
  }
}
Circle c = new Circle(42.0);
float s = c.area();
```

How to decide what is state and what is behavior?

[ Abstraction Rectangle Levels Rectangle Rectangle Function State FigureUtils Composition ]

# 4) What Is "Composition"?

```
1 canvas = new Canvas();
2 canvas.addCircle(new Circle(42));
3 canvas.draw();
```

```
1 canvas = new Canvas();
2 circle = new Circle(42);
3 circle.drawOn(canvas);
```

What is composition? What is the "right" composition?

Chapter #4:

# Object Thinking vs. Algorithms

# While-Do Loop

```
1  buffer = []
2  while true
3    c = STDIN.readchar
4    break if c == "\n"
5    if buffer.length > 3
6      STDOUT.puts buffer.join
7      buffer = []
8    end
9    buffer << c
10 end
```

```
1 $ echo 'Hello, world!' | ruby a.rb
2 Hell
3 o, w
4 orld
```

# Buffer Abstraction

```
buffer = []
while true
  c = STDIN.readchar
  break if c == "\n"
  if buffer.length > 3
    STDOUT.puts buffer.join
    buffer = []
  end
  buffer << c
end
```

```
class Buffer
  def initialize; @data = []; end
  def push(c)
    if @data.length > 3
      STDOUT.puts @data.join
      @data = []
    end
    @data << c
  end
end
buffer = Buffer.new
while true
  c = STDIN.readchar
  break if c == "\n"
  buffer.push c
end
```

# Loop Abstraction

```
1  class Buffer
2    def initialize; @data = []; end
3    def push(c)
4      if @data.length > 3
5        STDOUT.puts @data.join
6        @data = []
7      end
8      @data << c
9    end
10 end
11 buffer = Buffer.new
12 while true
13   c = STDIN.readchar
14   break if c == "\n"
15   buffer.push c
16 end
```

```
1  class Buffer
2    # the same
3  end
4  class Pull
5    def initialize(b); @buf = b; end
6    def again
7      c = STDIN.readchar
8      return false if c == "\n"
9      @buf.push c
10     true
11   end
12 end
13 buffer = Buffer.new
14 pull = Pull.new(buffer)
15 while pull.again; end
```

# Loop Abstraction

```
1  class Buffer
2    # the same
3  end
4  class Pull
5    def initialize(b); @buf = b; end
6    def again
7      c = STDIN.readchar
8      return false if c == "\n"
9      @buf.push c
10     true
11   end
12 end
13 buffer = Buffer.new
14 pull = Pull.new(buffer)
15 while pull.again; end
```

```
1  class Buffer
2    # the same
3  end
4  class Pull
5    # the same
6  end
7  class Pulls
8    def initialize(p); @pull = p; end
9    def fetch
10     while @pull.again; end
11   end
12 end
13 Pulls.new(Pull.new(Buffer.new)).fetch
```

# Object Composition

```
class Buffer
  def initialize; @data = []; end
  def push(c)
    if @data.length > 3
      STDOUT.puts @data.join
      @data = []
    end
    @data << c
  end
end

class Pull
  def initialize(b); @buf = b; end
  def again
    c = STDIN.readchar
    return false if c == "\n"
    @buf.push c
    true
  end
end

class Pulls
  def initialize(p); @pull = p; end
  def fetch
    while @pull.again; end
  end
end

Pulls.new(
  Pull.new(
    Buffer.new
  )
).fetch
```

Chapter #5:

# Enemies of Object Thinking

# What Makes Us Think as Algorithms

Static Methods

Anemic Objects (Getters)

Mutability (Setters)

Workers (“-er” Suffix)

NULL References

Type Casting (Reflection)

Inheritance

Chapter #6:

# Post-Test

> https://github.com/yegor256/hangman

# References


[1] Jeff Atwood. *Your Code: OOP or POO?* https://jttu.net/atwood2007. [Online; accessed 08-04-2025]. 2007.

[2] Yegor Bugayenko. *Alan Kay Was Wrong About Him Being Wrong*. https://www.yegor256.com/171212.html. [Online; accessed 08-07-2024]. Dec. 2017.

[3] Yegor Bugayenko. *What's Wrong With Object-Oriented Programming?* https://www.yegor256.com/160815.html. [Online; accessed 08-07-2024]. Aug. 2016.

[4] Bob Crawford. *A Series of Articles About Object Oriented Programming*. TUG LINES, 1989.

[5] Edsger W. Dijkstra. "The Humble Programmer". In: *Communications of the ACM* 15.10 (1972), pp. 859–866. DOI: 10.1145/355604.361591.

[6] Rich Hickey. *SE Radio 158: Rich Hickey on Clojure*. https://jttu.net/hickey2010. [Online; accessed 08-04-2025]. 2010.

[7] Alan Kay. *The Computer Revolution Hasn't Happened yet*. 1997. DOI: 10.1007/978-3-322-89884-5_3.

[8] Tim Korson and John D. McGregor. "Understanding Object-Oriented: A Unifying Paradigm". In: *Communications of the ACM* 33.9 (1990), pp. 40–60. DOI: 10.1145/83880.84459.

[9] Ole Lehrmann Madsen and Birger Møller-Pedersen. "What Object-Oriented Programming May Be — And What It Does Not Have to Be". In: *Proceedings of the European Conference on Object-Oriented Programming*. Springer. 1988, pp. 1–20. DOI: 10.1007/3-540-45910-3_1.

[10] Oscar Nierstrasz. *A Survey of Object-Oriented Concepts*. Citeseer, 1989.

[11] David Lorge Parnas. "On the Criteria to Be Used in Decomposing Systems into Modules". In: *Communications of the ACM* 15.12 (1972), pp. 1053–1058. DOI: 10.1145/361598.361623.

[12] Tim Rentsch. "Object Oriented Programming". In: *ACM SIGPLAN Notices* 17.9 (1982), pp. 51–57. DOI: 10.1145/947955.947961.

[13] Esther Schindler. *Linus Torvalds: Why C++ Sucks*. https://jttu.net/schindler2007. [Online; accessed 08-04-2025]. CIO, 2007.

[14] Asaf Shelly. *Flaws of Object Oriented Modeling*. https://jttu.net/shelly2015flaws. [Online; accessed 15-03-2016]. 2015.

[15] David West. *Object Thinking*. Pearson Education, 2004. DOI: 10.5555/984130.